# Andras Szasz - QA Engineer Home Assignment

## Task 1 – Manual Testing

Test case fields: ID, Title, Priority, Preconditions, Steps, Expected Result, Postconditions.

**Test-001: Add single product**
Preconditions: User is logged in and has an empty shopping list.
1. Navigate to a product page.
2. Click *Add to Shopping List* button.
3. Navigate to the *Shopping List*.

Expected Result: The product appears exactly once.

**Test-002: Add without login**
Preconditions: User is logged out.
1. Navigate to a product page.
2. Click *Add to Shopping List* button.
3. Navigate to the *Shopping List*.

Expected Result: Login prompt appears.

**Test-003: Add multiple products**
Preconditions: User is logged in and has an empty shopping list.
1. Navigate to a product page.
2. Click *Add to Shopping List* button.
3. Navigate to another product page.
4. Click *Add to Shopping List* button.
5. Navigate to the *Shopping List*.

Expected Result: The 2 products appears once each.

**Sample Bug Report**
Title: Duplicate items added when *Add to Shopping List* is clicked multiple times rapidly.
Severity: Major
Priority: High
App Version: 1.1
Browser Type: Chrome
Browser Version: 56.0
Expected: One item added.
Actual: Multiple duplicates appear.
Possible cause: Missing debounce.
Description: As a logged in user when the Add to Shopping List button is clicked multiple times rapidly then multiple duplicates gets added to the shopping list while the expected behaviour would be to limit the requests in a specific time interval (debounce) so that users won't accidentally add multiple items at once.

# Task 2 – Frontend Testing

I would design a multi-layered Playwright framework (which can also be used for API testing).

- Core: Where the framework's core functionalities lie alongside with additional fixtures and utilities.
- Test Data: Every data that the test cases require (ids, usernames, emails).
- Test Cases: Separated from the framework and the test data. Simple, easy to understand test cases.

This separation would provide maintainability, scalability, parallel development by multiple QA engineers.

**Prerequisites for running tests:**

- NodeJS: LTS version including NPM.
- Git: Version control (cloning the repo).
- VSCode: IDE for development (recommended).
- Browsers: Chrome, Edge, Firefox.
- Playwright: The framework itself.

**Sample Login Tests:**

```
import { test, expect } from “@playwright/test”;
test(“Successful Login”, async ({page})=>{
 await page.goto(“https://aldi.us/login");
 await page.getByLabel("Email").fill("user@test.com");
 await page.getByLabel(“Password”).fill(“Password123”);
 await page.getByRole(“button”,{name:”Login”}).click();
 await expect(page).toHaveURL(“/dashboard”);
});

test(“Invalid Password”, async ({page})=>{
 await page.goto(“https://aldi.us/login”);
 await page.getByLabel(“Email”).fill(“user@test.com");
 await page.getByLabel(“Password”).fill(“WrongPassword”);
 await page.getByRole(“button”,{name:”Login”}).click();
 await expect(page.getByText(“Invalid username or password”)).toBeVisible();
});
```

# Task 3 – API Testing

For API testing I would use Playwright's APIRequestContext because it integrates well with the existing UI framework and allows UI and API tests to share the same configuration, authentication, reporting and CI pipeline. Instead of placing HTTP requests directly inside the test cases, I would organise the framework into reusable layers.

- Core: Having the API client classes (BaseAPIClient, TaskClient), models (Task), utilities, fixtures.
- Test Data: Every data that the tests need in serialised format (tasks.json).
- Test Cases: The test cases themselves separated from the data and the core layer.

**BaseAPIClient:** Would centralise common HTTP functionalities such as request execution, headers, authentication, logging, error handling.

**TaskClient:** Would contain business-specific API calls so that if an endpoint changes, only this client needs to be updated instead of every test.

```
export class TaskClient extends BaseApiClient {
    async createTask(task) {
        return this.request.post("/tasks", {
            headers: this.defaultHeaders(),
            data: task
        });
    }
    async getTask(id) {
        return this.request.get("/tasks/${id}");
    }
    async updateTask(id, task) {
        return this.request.put("/tasks/${id}", {
            headers: this.defaultHeaders(),
            data: task
        });
    }
    async deleteTask(id) {
        return this.request.delete("/tasks/${id}");
    }
}
```

**Test Data** layer means no hardcoded values and less redundancy inside the test cases.

```
{
    "newTask": {
        "title": "Buy milk",
        "completed": false
    },
    "updatedTask": {
        "title": "Buy bread",
        "completed": true
    }
}
```

**Test Suite:** The lifecycle of a task naturally forms a CRUD workflow.

```
import { test, expect } from '@playwright/test';
import tasks from '../test-data/tasks.json';
import { TaskClient } from '../core/api/TaskClient';
let taskId: number;
test.describe('Task Management API', () => {
    let client: TaskClient;
    test.beforeEach(async ({ request }) => {
        client = new TaskClient(request);
    });
}
```

**POST /tasks**

```
test("Create a new task", async () => {
    const response = await client.createTask(tasks.newTask);
    expect(response.status()).toBe(201);
    const body = await response.json();
    expect(body.id).toBeDefined();
    expect(body.title).toBe(tasks.newTask.title);
    expect(body.completed).toBeFalsy();
    taskId = body.id;
});
```

**GET /tasks/{id}**

```
test("Get task by id", async () => {
    const response = await client.getTask(taskId);
    expect(response.status()).toBe(200);
    const body = await response.json();
    expect(body.id).toBe(taskId);
    expect(body.title).toBe(tasks.newTask.title);
    expect(body.completed).toBe(false);
});
```

**PUT /tasks/{id}**

```
test("Update task by id", async () => {
    const response = await client.updateTask(
        taskId,
        tasks.updatedTask
    );
    expect(response.status()).toBe(200);
    const body = await response.json();
    expect(body.title).toBe(tasks.updatedTask.title);
    expect(body.completed).toBe(true);
});
```

**DELETE /tasks/{id}**

```
test('Delete task', async () => {
    const response = await client.deleteTask(taskId);
    expect(response.status()).toBe(204);
});
```

**Verification:** Deleting a resource should always get verified.

```
test("Deleted task cannot be retrieved", async () => {
    const response = await client.getTask(taskId);
```

```
    expect(response.status()).toBe(404);
});
```

**Negative test cases:** A complete API suite should also verify error handling.

**Get non-existing task**

```
test("Get invalid task", async () => {
    const response = await client.getTask(999999);
    expect(response.status()).toBe(404);
});
```

**Expected Status Codes**

| Endpoint | Success | Common Error Cases |
|---|---|---|
| POST /tasks | 201 Created | 400 Bad Request |
| GET /tasks/{id} | 200 OK | 404 Not Found |
| PUT /tasks/{id} | 200 OK | 400 Bad Request, 404 Not Found |
| DELETE /tasks/{id} | 204 No Content | 404 Not Found |

**Reporting and CI**

The API suite should produce HTML reports, execution logs, and request/response details for failed tests. In a CI pipeline, the tests would run automatically on every pull request, with failures preventing merge.

Why this?

This structure separates business logic from test implementation, supports reuse through dedicated API client classes, and keeps tests maintainable. Separating test data makes scenarios easier to update, while adding both positive and negative cases provides bigger coverage of the API contract. It also scales well as the number of endpoints grows and integrates cleanly into CI/CD pipelines.

# Bonus

### Docker

Docker is a containerisation platform that packs an application together with its runtime, dependencies, and configuration into a lightweight, portable container. Therefore we can make sure that the application behaves consistently regardless of where it is executed. From a QA perspective, Docker helps eliminate the common “works on my machine” problem.
For a Playwright-based test framework, I would use the official Playwright Docker image.